# Accelerating Large-Scale Decision Tree Induction on Heterogeneous CPU-GPU Architectures

Bruno Mirčevski[0009−0007−1008−6708] and Krzysztof Jurczuk[0000−0001−6469−1769]

Faculty of Computer Science, Bialystok University of Technology, Bialystok, Poland

**Abstract.** We investigate parallel implementations of two widely used decision tree induction algorithms, CART and C4.5, on multicore CPUs, CUDA-enabled GPUs, and heterogeneous CPU-GPU architectures.

**Keywords:** Decision trees · CART · C4.5 · Parallel computing

## 1 Introduction

## 2 Related Work

### 2.1 Decision Tree Induction: CART and C4.5

*[Brief: CART [2] (Gini, binary splits, cost-complexity pruning), C4.5 [5] (gain ratio, MDL threshold cost, error-based pruning). Only what is needed to read Sect. 3.]*

### 2.2 Scalable and Parallel Induction on CPUs

### 2.3 Decision Trees on GPUs

## 3 Our Implementation and Optimizations

### 3.1 Simple

### 3.2 Serial

### 3.3 Parallel

### 3.4 GPU

### 3.5 Hybrid

## 4 Experiment Design

### 4.1 Datasets

*[One sentence per dataset (domain, how the test set is defined). All features numeric; ECBDL's nominal features as integer codes. The GPUs train ECBDL on its first 15M rows and SOREL on its first 4M (about 9.5G values each, 73 GiB of the 80 GB VRAM).]*

**Table 1.** Datasets. Sizes are of the training set as float32.

| Dataset | Features | Classes | Training rows | Test rows | Size | Test set |
|---|---|---|---|---|---|---|
| SUSY [1] | 18 | 2 | 4,500,000 | 500,000 | 0.3 GB | UCI split |
| HIGGS [1] | 28 | 2 | 10,500,000 | 500,000 | 1.2 GB | UCI split |
| ECBDL'14 [6] | 631 | 2 | 31,992,921 | 2,897,917 | 80.8 GB | official |
| GPU sample | 631 | 2 | 15,000,000 | 2,897,917 | 37.9 GB | official |
| MNIST8M [4] | 784 | 10 | 8,100,000 | 10,000 | 25.4 GB | MNIST test |
| SOREL-20M [3] | 2381 | 2 | 12,699,013 | 4,195,042 | 120.9 GB | official |
| GPU sample | 2381 | 2 | 4,000,000 | 4,195,042 | 38.1 GB | official |

### 4.2 Hardware

*[Four nodes of the mordor3 cluster (Table 2), each used whole (exclusive jobs). GPU nodes run the CUDA backend only, CPU nodes the CPU backends and the other CPU implementations. GPU runs are bound to the GPU's NUMA node (16 CPUs on hopper, 24 on ampere). Add measured memory bandwidth (STREAM) per node. Software: Ubuntu 24.04, Linux 7.0, g++ 13.3 (`-O3`), CUDA 12.9, NVIDIA driver 615.71.]*

**Table 2.** Hardware. Every node has two sockets with SMT disabled; CPU, cores and L3 are per socket. Clock is base/max turbo in GHz. One GPU (PCIe) is used per run.

| Node | CPU (per socket) | Cores | Clock | L3 | RAM | GPU |
|---|---|---|---|---|---|---|
| hopper | Xeon Gold 6444Y | 16 | 3.6/4.0 | 45 MB | 1 TB DDR5-4800 | H100 80 GB |
| ampere | EPYC 7402 | 24 | 2.8/3.35 | 128 MB | 1 TB DDR4-3200 | A100 80 GB |
| epyc1 | EPYC 9255 | 24 | 3.25/4.3 | 128 MB | 512 GB DDR5-6400 | – |
| dell1 | Xeon Gold 6542Y | 24 | 2.9/4.1 | 60 MB | 1 TB DDR5-5200 | – |

### 4.3 Compared Implementations

*[./tree backends (`-simple`, `-serial`, `-parallel`, `-cuda`, `-gpu-only`); scikit-learn and Weka J48 on the CPU; cuML and XGBoost on the GPU, with their settings. Why each is the closest equivalent (no public GPU C4.5; J48 depth limit adapter; XGBoost squared error = Gini on bins). rpart and YaDT: add or say why not.]*

### 4.4 Protocol and Measurement

*[One tree, depth $\leq$ 10, no pruning, all rows and features; what is timed (training only, from data in memory; GPU setup and upload included); peak RAM and VRAM; runs per case, means and spread; exclusive nodes; fairness (CPU threads of the hybrid, warm library processes, input layout). Source: docs/GPU-LIBRARIES.md, bench/README.md.]*

# 5 Results

## 5.1 Training Time

*[Overview of every implementation on every dataset (Tables 3 and 4): speed-ups of Serial-Opt over Serial-Simple, Parallel over Serial-Opt, GPU over Parallel, and of this work over the existing implementations. Variance: report mean with min–max, or mean ± 95 % confidence interval, once every cell has 10 runs.]*

**Table 3.** Training time of one CART tree (depth $\leq 10$, no pruning), in seconds: mean, and below it the fastest–slowest run. † marks a rough estimate (no run yet) with a plausible range below it; – not possible (data larger than the GPU's 80 GB, or no such algorithm). CPU runs on epyc1 (Parallel 48 threads, the rest one thread), GPU runs on the H100 (Hybrid with 16 CPU threads). Placeholder: every cell will have at least 10 runs.

| **This work** Dataset | Serial-Simple | Serial-Opt | Parallel | GPU | Hybrid |
|---|---|---|---|---|---|
| SUSY | 70.8<br>70.8–70.8 | 2.84<br>2.81–2.87 | 0.239<br>0.215–0.258 | 0.082<br>0.072–0.084 | 0.084<br>0.077–0.093 |
| HIGGS | 236<br>236–237 | 10.4<br>10.4–10.5 | 0.607<br>0.579–0.633 | 0.150<br>0.147–0.153 | 0.151<br>0.145–0.156 |
| ECBDL'14 | 5 860 | 503<br>500–506 | 13.5<br>13.5–13.6 | – | – |
| 15M sample | 2 270 | 225<br>225–225 | 6.32<br>6.29–6.34 | 2.97<br>2.96–2.98 | 2.97<br>2.96–2.98 |
| MNIST8M | 1 880 | 234<br>233–235 | 5.92<br>5.90–5.94 | 3.57<br>3.54–3.57 | 3.56<br>3.54–3.57 |
| SOREL-20M | 12 500 | 878 | 21.8<br>21.2–22.2 | – | – |
| 4M sample | 2 920 | 234<br>234–234 | 6.43<br>6.40–6.48 | *3.5*†<br>2.5–6 | *3.5*†<br>2.5–6 |
| **Existing implementations** Dataset | scikit-learn | rpart | | cuML | XGBoost |
| SUSY | 86.5<br>86.2–87.1 | *140*†<br>90–250 | | 0.133<br>0.126–0.147 | 1.43<br>1.42–1.45 |
| HIGGS | 215<br>214–215 | *350*†<br>200–600 | | 0.456<br>0.456–0.456 | 3.43<br>3.29–3.58 |
| ECBDL'14 | *4 500*†<br>3 000–9 000 | *7 500*†<br>4 000–15 000 | | – | – |
| 15M sample | 2 200 | *3 600*†<br>2 000–6 000 | | 9.95<br>9.90–10.0 | 42.2<br>42.1–42.4 |
| MNIST8M | *2 500*†<br>1 500–5 000 | *4 000*†<br>2 000–8 000 | | *7*†<br>4–14 | *50*†<br>25–100 |
| SOREL-20M | *13 000*†<br>7 000–26 000 | *21 000*†<br>10 000–50 000 | | – | – |
| 4M sample | *3 500*†<br>2 000–7 000 | *6 000*†<br>3 000–15 000 | | *10*†<br>5–20 | *45*†<br>25–90 |

## 5.2 Comparison with Other CPU Implementations

*[./tree's backends against scikit-learn (CART) and Weka J48 (C4.5): time, memory, test accuracy, same trees.]*

**Table 4.** Training time of one C4.5 tree (depth $\leq 10$, no pruning), in seconds: mean, and below it the fastest–slowest run. † marks a rough estimate (no run yet) with a plausible range below it; – not possible (data larger than the GPU's 80 GB, or no such algorithm). CPU runs on epyc1 (Parallel 48 threads, the rest one thread), GPU runs on the H100 (Hybrid with 16 CPU threads). Placeholder: every cell will have at least 10 runs. cuML grows an entropy tree, its closest equivalent.

| **This work** | | | | | |
|---|---|---|---|---|---|
| Dataset | Serial-Simple | Serial-Opt | Parallel | GPU | Hybrid |
| SUSY | 77.2<br>77.1–77.2 | 3.10<br>3.06–3.12 | 0.249<br>0.221–0.308 | 0.109<br>0.094–0.114 | 0.112<br>0.108–0.116 |
| HIGGS | 257<br>257–257 | 11.9<br>11.9–12.0 | 0.590<br>0.568–0.628 | 0.164<br>0.159–0.171 | 0.170<br>0.167–0.172 |
| ECBDL'14 | 6 890 | 623 | 17.6<br>17.5–17.7 | – | – |
| 15M sample | 2 590 | 277<br>277–277 | 7.81<br>7.76–7.86 | 2.99<br>2.99–3.00 | 2.99<br>2.99–3.00 |
| MNIST8M | 1 970 | 209<br>209–209 | 5.64<br>5.60–5.67 | 3.71<br>3.69–3.72 | 3.68<br>3.66–3.69 |
| SOREL-20M | 12 900 | 1 130 | 28.8<br>28.6–29.0 | – | – |
| 4M sample | 3 090 | 288<br>288–288 | 7.85<br>7.74–7.97 | *3.5*†<br>2.5–6 | *3.5*†<br>2.5–6 |

| **Existing implementations** | | | |
|---|---|---|---|
| Dataset | Weka J48 | YaDT | cuML |
| SUSY | 230<br>229–230 | *46*†<br>25–80 | 0.128<br>0.119–0.139 |
| HIGGS | 830 | *170*†<br>90–300 | 0.433<br>0.414–0.457 |
| ECBDL'14 | *50 000*†<br>25 000–100 000 | *10 000*†<br>5 000–20 000 | – |
| 15M sample | 24 200 | *4 800*†<br>2 500–9 000 | 9.89<br>9.84–9.98 |
| MNIST8M | *14 000*†<br>7 000–28 000 | *2 700*†<br>1 200–6 500 | *7*†<br>3–13 |
| SOREL-20M | *60 000*†<br>30 000–120 000 | *12 000*†<br>5 000–30 000 | – |
| 4M sample | *18 000*†<br>9 000–40 000 | *3 800*†<br>1 500–9 000 | *10*†<br>5–20 |

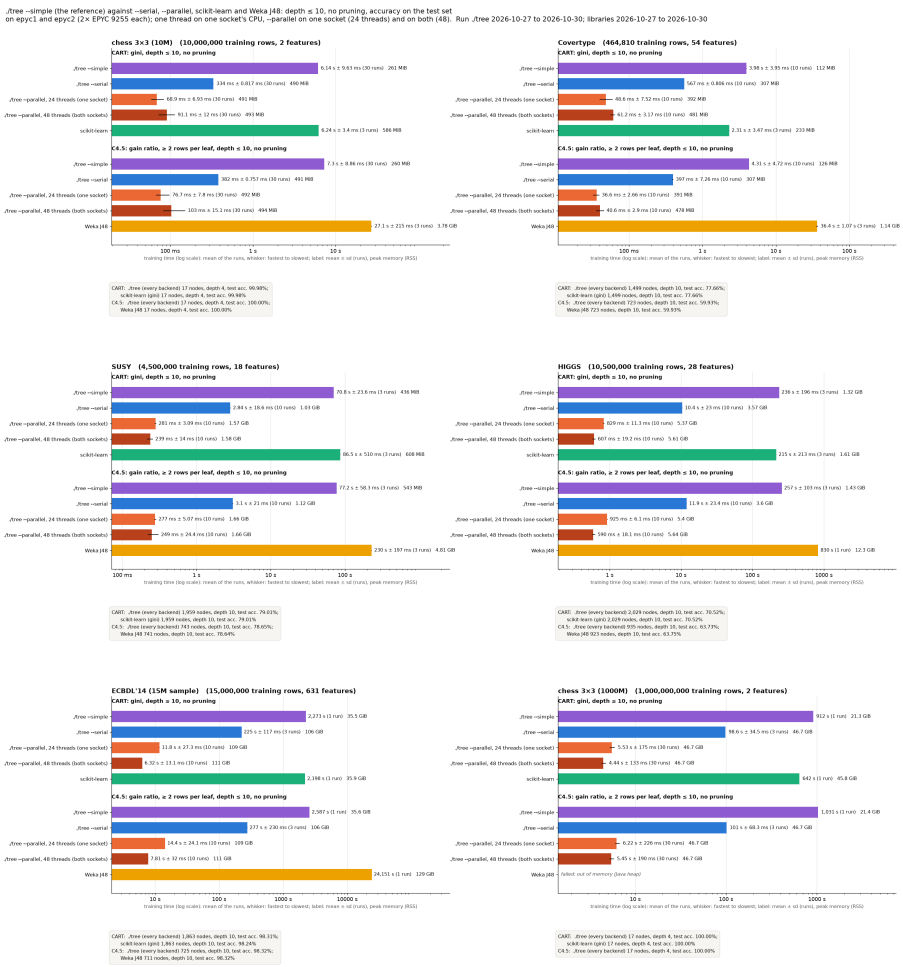



**Fig. 1.** Temporary: ./tree against scikit-learn and Weka J48 (cluster-cpu-vs-others).

### 5.3 Comparison with GPU Libraries

*[Against cuML and XGBoost: one tree, and per tree of a 10-tree forest; accuracy of exact versus binned trees; VRAM.]*

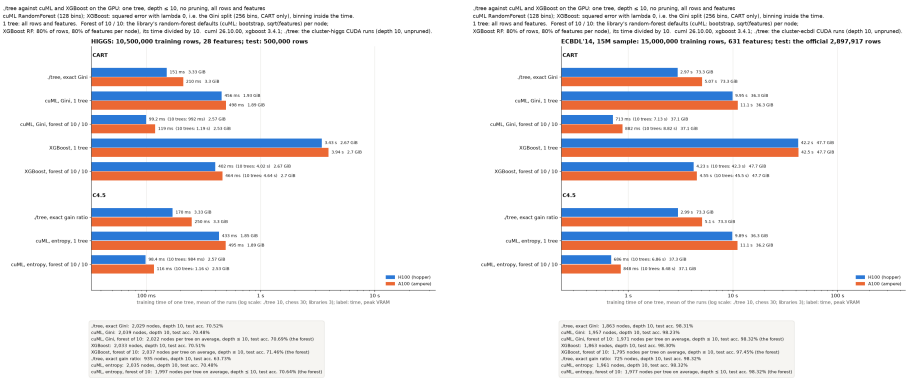



**Fig. 2.** Temporary: ./tree against cuML and XGBoost on HIGGS (left) and ECBDL'14 15M (right) (cluster-gpu-vs-others).

### 5.4 Performance Across Hardware

*[All ./tree backends on the four nodes, per dataset: serial, one and two sockets, H100 and A100 hybrid and GPU only. CPU against GPU; which hardware wins for which data shape (rows, features, classes).]*

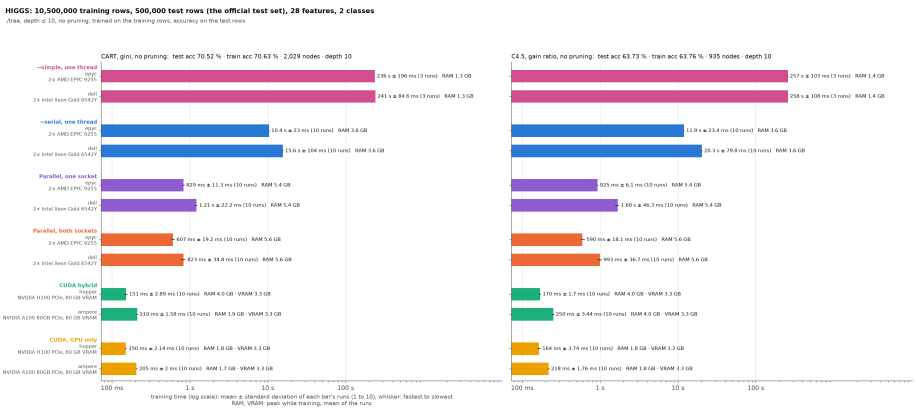



**Fig. 3.** Temporary: ./tree's backends on HIGGS on every node (cluster-higgs).

## 5.5 Scalability

*[Thread scaling 1–48 on epyc1 and dell1 (memory bandwidth bound); growth with the number of rows (subsamples of one dataset).]*

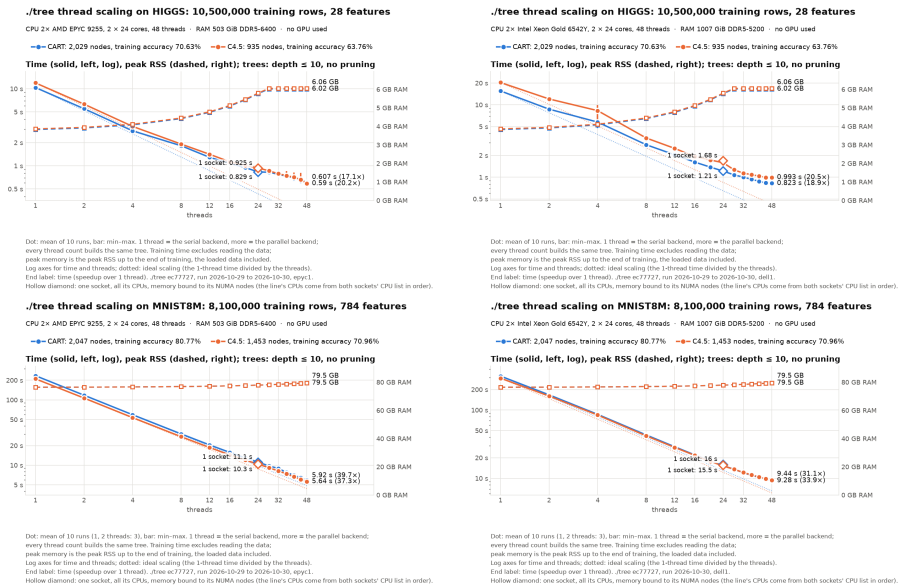



**Fig. 4.** Temporary: thread scaling on HIGGS and MNIST8M (cluster-cpu-scaling).

## 5.6 Profiling

*[Where the time goes: presort, split search, partition, CPU–GPU transfer; GPU share of the hybrid; achieved bandwidth against peak.]*

## 5.7 Energy Consumption

*[Whole-server power while each implementation trains one tree on ECBDL'14 15M (the one dataset every implementation runs on, GPUs included). Method: power source and sampling rate (BMC/IPMI for the server, RAPL for the CPUs, `nvidia-smi` for the GPU), idle power measured before the runs and subtracted, energy = integral of power over the training time, 10 runs each. The CPU implementations run on a CPU node, the GPU ones on a GPU node, so the totals compare servers, not only processors. Discuss energy per tree: a fast run at high power can still use the least energy.]*

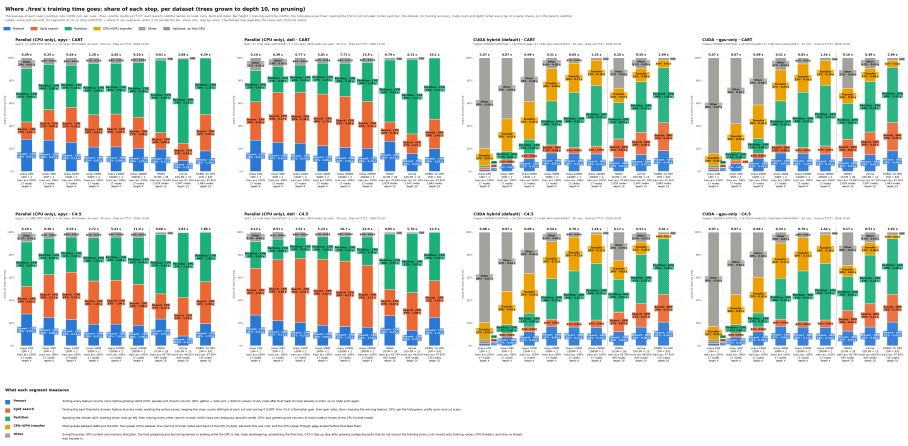


**Fig. 5.** Temporary: breakdown of the growing time (cluster-profiling).

**Table 5.** Energy of one tree on ECBDL'14 15M (depth $\leq$ 10, no pruning): mean training time $t$, mean power $P$ of the whole server above idle, and energy $E = P \cdot t$. Placeholder: no measurements yet.

| | | CART | | | C4.5 | | |
|---|---|---|---|---|---|---|---|
| Implementation | Node | $t$ (s) | $P$ (W) | $E$ (kJ) | $t$ (s) | $P$ (W) | $E$ (kJ) |
| Serial-Simple | epyc1 | ? | ? | ? | ? | ? | ? |
| Serial-Opt | epyc1 | ? | ? | ? | ? | ? | ? |
| Parallel | epyc1 | ? | ? | ? | ? | ? | ? |
| GPU | hopper | ? | ? | ? | ? | ? | ? |
| Hybrid | hopper | ? | ? | ? | ? | ? | ? |

## 6 Discussion

## 7 Conclusion

## References

1. Baldi, P., Sadowski, P., Whiteson, D.: Searching for exotic particles in high-energy physics with deep learning. Nature Communications **5**, 4308 (2014)
2. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth, Belmont, CA (1984)
3. Harang, R., Rudd, E.M.: SOREL-20M: A large scale benchmark dataset for malicious PE detection (2020), arXiv:2012.07634
4. Loosli, G., Canu, S., Bottou, L.: Training invariant support vector machines using selective sampling. In: Large Scale Kernel Machines, pp. 301–320. MIT Press (2007)
5. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo, CA (1993)
6. Triguero, I., del Río, S., López, V., Bacardit, J., Benítez, J.M., Herrera, F.: ROSEFW-RF: The winner algorithm for the ECBDL'14 big data competition. Knowledge-Based Systems **87**, 69–79 (2015)